

ShopAssist Commerce Support Desk - Design Decisions

1. Goal

The goal of this project is to build a simple but professional support system for an online store. The system accepts a customer question, checks available order or product data, calls the correct tools, and returns a clear customer-ready answer.

The required public function is kept exactly as requested:

```
run_agent(question: str) -> str
```

The latest version separates public support features from backend operations. The public support website can answer questions and show read-only order information, while fulfillment status updates are handled only from a protected admin backend panel.

2. Architecture

The project has four clean layers.

Core Assistant Layer

The core logic is inside the `shopassist_agent` package. This layer contains:

- `agent.py` for request routing and final answer generation
- `nlu.py` for request type detection and entity extraction
- `tools.py` for required backend functions
- `data.py` for mock store data
- `schemas.py` for structured response objects
- `logger.py` for JSONL activity logging

This separation keeps the assignment logic easy to test and easy to explain.

Backend API Layer

The FastAPI backend is in `backend/main.py`. It exposes endpoints for:

- assistant replies
- assignment-compatible answer-only response
- read-only order details
- protected admin order list
- protected admin fulfillment status updates
- product details
- product search
- dashboard metrics
- activity logs
- health checks

FastAPI was selected because it provides automatic Swagger documentation, request validation, clean endpoint structure, and easy deployment support.

Public Support UI Layer

The public Streamlit interface in `app.py` provides:

- customer message input
- support reply display
- resolution timeline
- catalogue dashboard
- read-only order tracking overview
- activity logs
- backend mode indicator

The public interface intentionally does not include any status update controls. This makes the UI more realistic because general users or support viewers cannot edit fulfillment records.

Private Admin Panel Layer

The private Streamlit admin panel in `admin_panel.py` provides:

- admin key login
- protected order list
- editable fulfillment status
- tracking ID update
- courier partner update
- delivery ETA update
- customer-facing shipment note update

The admin panel calls the protected backend endpoint with an `X-Admin-Key` header.

3. Tool Selection Logic

The project preserves the three required tools:

```
get_order(order_id)
search_products(query)
get_product(product_id)
```

The assistant selects tools based on the customer message:

- Order tracking questions call `get_order`.
- Product detail questions call `get_product`.
- Search or recommendation questions call `search_products`.
- Cheaper alternative questions may chain multiple tools.

Example customer question:

```
Is there a cheaper alternative to the shoes I ordered in ORD-1002?
```

The system performs:

1. `get_order("ORD-1002")` to identify the purchased product.
2. `get_product(product_id)` to fetch product details.
3. `search_products(category)` to find relevant alternatives.
4. It filters cheaper and in-stock products.
5. It returns a customer-friendly answer.

4. Error Handling

The system avoids guessing. If the customer provides an invalid order ID, the answer clearly says that the order was not found. If a product ID is invalid, it says the product was not found. If search results are empty, the assistant asks the customer to try another category or budget.

The API layer also returns appropriate HTTP responses:

- Invalid order endpoint: 404 Not Found
- Invalid product endpoint: 404 Not Found
- Invalid fulfillment status: 400 Bad Request
- Missing or wrong admin key: 401 Unauthorized
- Empty question body: request validation error

This makes the backend more professional and easier to test.

5. Customer-Friendly Responses

The tools return structured data, but the customer never sees raw tool output. The assistant converts internal data into clear support messages that include useful details such as product name, order status, delivery estimate, carrier, price, stock, and rating.

Example:

Your order ORD-1002 is currently Shipped. Items: AeroFlex Running Shoes x 1. Expected by 2026-06-28. La

6. Backend Design

The backend includes:

- /health endpoint for deployment checks
- /docs Swagger UI for API testing
- /api/agent/ask for full support result with trace
- /api/agent/simple for answer-only assignment compatibility
- /api/orders/{order_id} for read-only order lookup
- /api/order-statuses for supported fulfillment stages
- /api/admin/orders for protected admin order access
- PATCH /api/admin/orders/{order_id}/status for protected status updates
- /api/products/search for catalogue search
- /api/metrics for dashboard metrics
- /api/logs for recent activity
- CORS support so a frontend can call the API

The protected status endpoint modifies the current in-memory order store and logs the change. In a production version, the same endpoint can write to a SQL/NoSQL database, trigger courier API updates, or notify an order-management system.

7. Fulfillment Status Update Flow

Supported statuses are:

Processing, Dispatched, Shipped, Out for Delivery, Delivered, Cancelled

The update flow works as follows:

1. An operations user opens `admin_panel.py`.
2. The user enters the admin API key.
3. The user selects an order.
4. The user chooses a new status such as `Dispatched` or `Delivered`.
5. Optional tracking ID, carrier, ETA, and update note can be edited.
6. The admin panel sends a protected request to `PATCH /api/admin/orders/{order_id}/status`.
7. FastAPI validates the admin key and status value.
8. The backend updates the order record and logs the change.
9. The public support website remains read-only but displays the updated status.
10. The support assistant uses the latest status when the customer asks about that order.

This design shows backend write capability without exposing edit controls on the public website.

8. Logging

The project logs decisions and tool calls in JSONL format. This helps during debugging and evaluation because the reviewer can see which function was called and whether it succeeded.

The public UI shows recent support logs in the Activity tab. Status updates are also logged by the backend operation.

9. Testing

The project includes unit tests for both the support assistant and the API backend. Tests cover:

- order status questions
- cheaper alternative flow
- invalid orders
- invalid products
- product search
- budget filtering
- empty input
- API health check
- API agent response
- API product search
- API 404 behavior
- protected admin route authentication
- admin status update for `Dispatched` and `Delivered`
- invalid status validation

Verified result:

34 passed

10. Future Improvements

Possible future improvements include:

- connecting to a real SQL or NoSQL database
- adding proper user login with JWT or OAuth

- adding role-based access control for support and operations users
- adding order cancellation and refund workflows
- adding payment status lookup
- adding real courier tracking API
- using an LLM provider for more flexible language understanding
- deploying backend on Render/Railway and UI on Streamlit Cloud

11. Final Summary

This project satisfies the original assignment requirements while adding a professional backend, a protected admin operations panel, read-only public support UI, logging, tests, and a clean company-ready structure. The main focus is reliable tool selection, correct tool chaining, graceful error handling, customer-friendly replies, and realistic separation between public UI and backend operations.